

流程记忆方式：具体是做什么+分片上传和断点续传怎么实现

分片上传和断点续传

你简历中有写使用了分片上传和断点续传，那么讲一讲分片上传和断点续传是怎样的？

1. 分片上传要做的是将一个完整的视频拆成分片同时执行多次上传并在上传完成后合并分片成完整文件，断点续传是确保上传中断情况下重新上传能只上传未上传过的分片，而不上传已上传过的部分。

2. 原先实现方式是前端会将原始文件拆分成多个小分片，同时启动多个线程，每个线程都会对应一个分片，先发请求查询该文件的该分片之前是否已经上传过，响应通知上传过那么后续上传分片操作就不执行

3. 通知没上传过发请求上传分片和总的片数到后端，后端每次接收文件分片后用特定文件名做键，记录该文件已经上传的分片索引的集合做值，将文件已上传分片索引存到 `map` 中，并指定一个 `minio` 的地址，将分片上传到 `minio` 的这个地址，若接收完分片后发现文件已上传的分片数量等于总的分片数则执行合并操作，这样就能利用多线程同时上传多个分片加快文件上传速度，同时保证因为网络波动或其他原因导致的上传中断后重新上传分片时，同一个已上传分片不再上传。

4. 上传文件和保存新视频信息到数据库被拆成了两步，上传最后一个分片的请求得到的响应中会有合并后的视频存储在 `minio` 中的地址，用户将视频标签视频简介填写好、选择一个文件做封面，然后选择上传按钮将视频标签、简介、封面文件流、视频存储在 `minio` 中的地址发送给后端，后端将封面上传到 `minio` 并存储封面在 `minio` 中的地址，再将这些视频信息保存到 `mysql`，并移除 `map` 中该文件对应的键值对

5. 项目中使用 `mysql` 存储文件的话性能不高且上传下载需要做额外的转换，因此采取了对象存储服务，而 `minio` 可以用于存储文件对象，因此选择 `minio`，不选择 `oss` 的原因是 `minio` 无成本，只需要自己部署。

6. 但这种方案上传速度仍然可以优化，我的想法是后端可以用于提供一个安全凭证，上传文件时前端向后端发送请求获取一个可以临时和 `minio` 交互的令牌，用该令牌做凭证直接将分片上传到 `minio` 和执行合并，这样可以省去从前端传输到后端的时间，临时令牌也保证了上传权限不会被客户端长久持有，但上传文件内容无法在 `minio` 端进行解析，因此会有被上传恶意文件的可能，所以如果更看重性能可以选前端直接连 `minio`，更看重安全性可以用原项目方法。

辅助记忆

1. 分片上传和断点续传是要做什么

2. 分片上传前的查询

3. 查出分片没上传过后的操作

4. 补充完整数据库视频相关信息

5. 使用 `minio` 的原因

6. 其他优化方案

上传选择封面是强制的吗？如果是的话我现在改成不强制，应该怎么保证视频还是有封面？

是的。可以在上传环节中保存视频数据到数据库的接口里做检测，如果封面文件流是空的则用视频存储在 `minio` 中的地址去下载完整文件流，然后调用工具库剪切下来一帧图片作为封面文件。

分片的合并是怎么做的？

分片的合并是指定 minio 中几个分片文件的存储地址以及合并后完整文件存储的地址，远程通知 minio 端进行几个文件的合并，并将合并后的完整文件存储在指定的地方

而存储地址由 minio 桶的地址加文件名组成，视频文件存储的桶地址后端已经固定写好，分片文件按特定视频文件名加分片索引的格式命名，合并时从 map 中取某个视频的特定文件名加分片的索引得到分片文件名，加上桶地址得到分片存储地址，遍历后得到所有分片存储地址并指定合并后的完整文件存储地址为桶地址加特定文件名。

备注：minio 的桶类似 windows 的文件夹

前端上传设置几个线程数？

弱网环境下，减少并发数，避免因网络波动导致多个分片上传失败，增加重试成本。强网环境下可适当提高，且由于浏览器对同一域名的并发请求数有限制，一般为 6-8 个，因此弱网环境下设置为 3-4 个，强网环境下设置为 5-6 个，如果问为什么不设置满就说是为了扩展性，防止后续可能会有其他请求也在并发

你提到有特定文件名，那么该文件名如何生成？会出现两个不同的文件的特定文件名重合的情况吗？如果会的话如何应对？

由文件的修改时间加文件名组成。有可能出现，可以在特定文件名后再加随机生成的 uuid 或者加上原始文件的哈希值让不同的文件撞名字概率大大降低。

在分片上传过程中，如果某个分片上传失败，应该怎么处理？

分情况考虑，如果前端网络波动等原因导致前端无法将分片传递给后端，则前端恢复后执行断点续传流程

若连接不上后端则前端持续重试直到较多次数后放弃重试，通知用户上传失败

若是服务端上传 minio 过程失败则服务端进行对 minio 的上传重试

若多次重试仍失败则响应通知前端上传失败，保证只有所有分片都上传成功了才能执行下一步将视频信息和视频实际存储地址传给后端，保存新视频数据到 mysql 中。

还可以别的方式实现分片上传和断点续传吗？如何实现极速秒传？

1. 后端将前端传递的完整文件分片并启动多个线程上传到 minio，这种方式安全性更高，不暴露 minio 地址且可以先提前审查文件是否是恶意文件，但速度没有前端直传快

2. 或者前端分片后发送请求从后端获取一个临时凭证，然后携带这个凭证直接连接 minio 不经过后端中转，这样虽然会暴露 minio 地址，但可以通过对 minio 的设置来限制上传时必须带凭证且速度比较快，但无法审查出恶意文件

3. 断点续传则是可以客户端记录已上传分片的索引集合，而不是服务端记录，这样服务端存储压力更小，但客户端相比服务端可靠性可能会更低

4. 极速秒传则是给视频表加一列用来存视频的哈希值，每次上传视频之前会前端计算当前视频的哈希值出来，发送请求将值传给后端，这个请求对应的后端接口会查询视频表是否有相同哈希值的记录。

没有的话就正常上传并且把视频的哈希值存到视频表中，如果查询到有那么就在这个相同哈希值的视频的实际存储地址返回前端，并且页面上显示上传进度条拉满，直接完成上传。

辅助记忆

- 1.后端接收完整文件并分片上传以及优缺点
- 2.前端从后端拿凭证后直连 minio 以及优缺点
- 3.断点续传客户端记录分片上传状态
- 4.极速秒传实现

已上传分片的记录存储在内存 Map 中，若服务器重启，如何保证断点续传的可靠性？

内存 map 依赖于服务器，那么找到一个不依赖服务器的存储位置可以保证断点续传的可靠性，可以将已上传分片的记录存储位置改到 redis 中。

如果要加个视频下载功能，你如何设计？

可以让前端用初始化播放页面时，请求拿到的视频存储在 minio 的地址去下载完整文件，也可以简单的发送请求，后端从 minio 中获取对应视频的完整文件流再返回给前端
也可以再优化，后端获取完整文件流后分片返回前端，让前端进行聚合，提升下载速度
还可以再优化，将上传分片时前端传来的分片利用起来，这样的话新建一个表记录某个视频和该视频的分片实际存储 minio 中的地址
在某个分片上传后向该表中存储一条新记录绑定该视频和视频的一个分片实际存储地址，下载一个视频时从该表中查出对应的分片实际存储地址后多线程下载多个分片再将多个分片返回前端，由前端执行分片聚合。

有没有评估过在线播放支持多少人使用？

没有，但如果要评估的话思路是将 QPS 不断拉高，直到服务宕机，这个导致服务宕机的 QPS 数就是在线播放支持多少个人使用的上限。

假如有个视频网站原型，多人看视频加载不了，如何排查分析？

从加载视频的完整流程进行分析，前端发送请求到后端
后端查询数据库并得到视频存储在 minio 中的地址，将该地址返回前端
前端用该地址请求 minio 获取视频资源，若是前端请求后端失败则可能是后端服务宕机或后端接口出问题
若后端服务正常运行则看数据库是否连接失败了
若连接无误则查看数据库查询 sql 是否有误，若也无误则看数据库中存储的视频存储在 minio 中地址是否有误。

如果同时有特别多用户同时上传视频，导致 java 内存占用过多，你该怎么办？

可以让前端在将原始文件分片前先压缩原始文件降低文件大小
可以启动多个视频服务的实例做负载均衡来减低单个视频服务实例的负担
还可以控制上传视频数，设定一个值，最大只能上传多少个视频，当前正在上传的视频数到达这个值后就阻塞剩余的上传线程直到正在上传视频数减少到小于这个值。

同时上传过多视频导致服务器宕机怎么办？

宕机后需要运维来重启服务器，后端能做的则是提前设计预防宕机方案，可以在上传完一个视频后手动清理视频缓存，使内存占用更快被释放
可以让前端在将原始文件分片前先压缩原始文件降低文件大小
可以启动多个视频服务的实例，多个实例运行在不同的服务器上，做负载均衡来减低单个服

服务器的负担

还可以控制上传视频数，设定一个值，单个服务实例最大只能上传多少个视频，当前正在上传的视频数到达这个值后就阻塞剩余的上传线程直到正在上传视频数减少到小于这个值。

分片上传为什么可以解决上传慢的问题？

因为服务器的带宽往往有限，比如 2m 每秒，3m 每秒，所以拆成分片上传可以充分利用带宽，内网上传就优化效率不大，因为内网带宽基本都高，自己的电脑大概率是 50mb/s 往上。
备注：内网上传和公网上传的区别就是自己启动前后端然后在本机上传和前后端都部署在服务器，你只是进入前端的线上地址去上传视频到服务端。

怎么做 ai 视频总结？

找开源的库视频文件转音频，音频再转文字，然后将文字作为提问传给大模型 api，调用 api 获取视频总结。

视频编码考虑过吗？

考虑过，mp4 视频文件兼容性最高的编码是 h264，所以有设想过若有些 mp4 视频的编码不是 h264 且播放会出问题，可以调用 api 将编码转成 h264 来解决播放问题，不过实际测试中没有遇到过这种情况。

如果分片 1 上传到后端，后端分片上传接口还没处理完，前端就断网了，恢复后重试是否会出现查询分片 1 上传状态得到未上传过的响应进而重复上传了分片 1 的情况？

不会出现，因为分片 1 上传到后端后第一时间就会往 map 中添加这个分片 1 的索引，上传分片文件时也做了捕获异常，如果上传失败会将 map 中的分片索引移除，防止出现文件还有分片是上传失败状态就执行合并的情况。

如果我断点续传时想 1MB 都不浪费，之前已经上传到后端了的分片文件流也利用起来，比如分片 1 上传了一半，这时网络中断，正常情况是重新传分片 1 的完整文件，但我只想传剩下的一半，应该怎么办？

修改原有逻辑，新创建一个 map 和定义一个属性为文件流和字节数的自定义对象，在后端分片上传的接口中捕获到网络连接中断的异常后，用当前已经上传了的文件流和该文件流所具备的字节数创建一个自定义对象，然后将该自定义对象作为值，特定文件名后拼接上分片的索引作为键，存到 map 里，并新提供一个查询当前分片已经上传的字节数的接口。当前端断点续传时先查询当前分片是否已经上传过，如果结果是没上传过则查询当前分片已经上传的字节数，如果字节数为 0 则上传完整分片文件，如果不为 0 则前端剪切分片文件，到已上传完的字节数对应的文件流位置开始上传新的文件流，同时后端接口接收文件流时查询 map 中是否有当前分片对应的残留文件流，若无则按正常流程执行，若有则取出当前分片的文件流并将新的文件流拼接之前缓存的文件流末端，并在这次分片上传完成后删除 map 中对应的缓存文件流，减少内存占用。

分片损坏了怎么办？

分片损坏可能是前端上传到后端的分片本身就是坏的，也可能是前后端传输过程中损坏，也可能是从后端上传到 minio 的过程中损坏了，可以在前端上传分片时顺便上传分片哈希值，后端获取前端上传的分片后计算哈希值，如果和前端传的哈希值不同则让前端重新上传，以及上传到 minio 前先保存到本地的临时文件中，保存完后计算临时文件哈希值并和接收分片后的哈希值做对比，如果不一样则重新保存直到两者一致确保保存的临时文件是完好的，然

后再从临时文件上传到 minio，上传完成后从 minio 读取分片计算哈希值，如果不一致也重新读取临时文件再上传 minio 并重复校验直至哈希值一致。

为什么要保存到临时文件？

因为前端上传到后端的文件流是一次性的，上传完 minio 后无法第二次上传。也可以用 `ByteArrayInputStream` 这种可复用的文件流，前端分片上传后先将分片文件流保存到 `ByteArrayInputStream` 中，之后的流程 `ByteArrayInputStream` 替代临时文件的作用。

一对一实时私聊

说说基于服务端中转架构的一对一实时私聊如何实现的？

1. 一对一实时私聊是当发送消息和接收消息的双方同时在线时，发送方发出消息后，消息立刻就会显示在接收方聊天页面。
2. （原先）实现方式是每个客户端进聊天页面后向后端发送 `websocket` 连接申请，后端同意连接后会创建一个 `websocket` 对象代表这次连接，使用这个对象可以从后端向这次连接的客户端发送 `websocket` 消息
3. 建立连接后客户端会发送 `websocket` 消息，将当前客户端 `userid` 传递给后端，后端调用处理 `websocket` 消息的方法，将当前客户端的 `userid` 和代表这次连接的 `websocket` 对象存到 `map` 中，初始化步骤完成
4. 若后续有其他客户端要向当前客户端发送消息只需要将消息内容、其他客户端的 `userid` 和当前客户端的 `userid` 放到一起发送 `websocket` 消息给后端，后端调用处理 `websocket` 消息的方法，在 `map` 中查询当前客户端对应的 `websocket` 对象，查找到后用该对象从服务端向当前客户端发送 `websocket` 消息，从而实现消息的转发。并且每次有客户端的 `websocket` 消息进入后端都会保存聊天内容到数据库做持久化。
5. 当用户离开聊天页面后会关闭 `websocket` 连接，后端也会同步删除 `map` 中该用户对应的键值对，及时更新用户会话状态。
6. 我觉得还可以用 `http` 的形式，定时任务发送请求获取最新的聊天内容，将定时任务间隔时间设置的较短来达到近似实时通信的功能，但这种方式需要频繁发送请求，对服务器负担可能过重，或者用 `sse` 服务端单向推送的技术，整体流程类似 `websocket`，对服务端负担更小，但实现更复杂。

辅助记忆

1. 实现效果
2. 后端怎么向前端发送 `websocket` 消息
3. 前端向后端发送消息，完成初始化
4. 后续消息如何转发
5. 清理会话
6. 优化方案

如果我现在要扩展成多对多的实时私聊，应该如何实现？

1. 创建消息表、群聊表和关联用户与群聊表的中间表，消息表有群聊 `id` 和发送人 `id` 和消息内容字段，群聊表有群聊名称群聊 `id`，关联中间表有群聊 `id` 和用户 `id` 字段

2.进入页面时会发请求拉取群聊列表，用户增删改群聊会对群聊表进行修改，当用户需要加入某个群聊时，可以通过输入群号或群名称的形式先搜寻到群，再选择加入，然后对用户群聊中间表进行新增

3.有新消息发送到某个群时则往消息表新增记录，最后客户端执行较短间隔的定时任务定期从服务端拉取用户所在群的群聊消息和群聊状态来更新页面。

4.如果是 **websocket** 的形式则在进入聊天页面时建立 **websocket** 连接，并拉取当前用户的群聊列表，还是需要建立前面的表，新增群聊则发送请求更新数据库，同时新增修改删除群聊的 **websocket** 消息类型，当用户发送修改删除群聊的携带有群 id 的 **websocket** 消息时将消息发送给在该群且在线的其他客户端来实时同步状态，同时持久化群聊情况到数据库当用户需要加入某个群聊时，也是通过输入群号或群名称的形式先搜寻到群，再选择加入，并新增新入群群的 **websocket** 消息类型，有新入群的用户时发送 **websocket** 消息给服务端，服务端同步给在该群且在线的其他客户端，同时持久化到数据库同理，新增群聊消息的 **websocket** 消息类型，若有聊天消息则服务端将消息转发给在该群且在线的其他用户并持久化到数据库。

辅助记忆

1.创建的表和表字段

2.群聊增删改查以及加入群聊的实现

3.群聊消息发送以及状态更新实现

4.如果是 **websocket** 形式怎么实现前面流程

是用的什么进行的 **websocket** 连接管理？如果服务重启，如何恢复用户会话？

Java 的 **hashmap**。由于服务端无法主动向客户端发起 **websocket** 连接，可以客户端做心跳机制，定时向后端发送 **websocket** 消息，超过一定时间得不到响应就重新向后端申请建立 **websocket** 链接。

如果客户端异常断连，服务端如何主动清理残留的 **Session** 对象？

定时任务定期检查使用 **map** 中的 **websocket** 对象发送消息会不会报错，如果会的话那就说明连接断开了，或者在某一次转发 **websocket** 消息时监测到连接断开，那么将异常的 **websocket** 对象所对应的键值对进行删除。

在使用服务端中转的架构设计实现用户一对一实时私聊时，如何避免消息丢失？

后端转发 **websocket** 消息的同时将私聊内容持久化到 **mysql** 数据库。

若要为私聊功能添加消息撤回功能，请你给出实现思路？

Websocket 消息在原先的初始化类型、大模型类型、聊天内容类型之外加一种撤回类型同时每条消息都绑定一个 **id**，一端选择撤回时也发送 **websocket** 消息给后端，根据该消息的 **id** 删除数据库中的记录，并转发一条撤回消息给另一个客户端，让另一个客户端页面上删除对应 **id** 的聊天消息。

聊天为什么要在服务端进行一次中转，而不是直连？

相比直连消息只在客户端存储，服务端中转能让消息在服务端持久化存储，丢失概率低，对

客户端网络要求更低，且相比直连实现更简单，但延迟会更高一些，而项目中的私聊不需要特别高的实时性，因此选择服务端中转。

私聊如何保证消息有序性？

1. 私聊出现乱序的情况应该是部署了多个聊天服务实例的情况下

比如两个实例负担不一样重，先发出去的聊天消息进入了负担重的实例，后发出去的聊天消息进入了负担轻的实例，负担轻的实例处理速度更快，导致后面的聊天消息反而先存入数据库，后面的聊天消息反而先转发给客户端

2. 可以在 gateway 服务中建立一个全局的 websocket 过滤器和一个记录每个用户所分配实例的键为用户 id 值为实例名的 map，每次有 websocket 消息过来时，用该 websocket 消息中的用户 id 去 map 中查询是否已有绑定的聊天服务实例

若有则直接转向该实例，若无则随机选择一个实例转向，并往 map 中新增记录，用这种形式实现同一个用户的 websocket 消息只由同一个实例处理，进而保证消息有序性。

辅助记忆

1. 出现乱序的原因

2. 用 gateway 建立全局过滤器保证同一个用户的信息由同一个实例处理

有大量用户在使用私聊功能的情况如何应对？

1. 启动多个私聊服务做负载均衡来增强私聊功能的可承受并发量

2. 若不修改服务器配置的情况可以修改 springboot 线程池配置，项目中接收 websocket 消息后的高频事件是将聊天消息插入数据库和转发消息给另一个客户端，属于 I/O 密集型，因此可以配置核心线程数为 $n*2$ 来提升线程池效率

3. 同时可以先测试临界值，看后端 1s 内的 websocket 消息处理数是多少时服务器会宕机

4. 改造原有 websocket 消息，在原先发送 websocket 消息步骤前加一条 websocket 消息的发送，这条消息负责在后端查询当前有多少个正在处理的 websocket 消息并返回客户端，若客户端得到的值大于等于临界值则先阻塞 websocket 消息的发送，这样可以避免流量过大服务器宕机

并轮询查询该值直到该值小于特定值，将阻塞的前端线程解开，发送 websocket 消息出去，而后端每次处理一条 websocket 消息前先给当前处理数加一，同时启动一个定时任务每秒将当前处理数重置为 0。

5. 聊天消息的持久化也可以不在每次有聊天消息类型 websocket 消息来时进行，可以缓存起来然后定期批量保存到数据库提升效率。

以及合理设置连接的超时时间，对于长时间无活动的连接进行及时清理，释放资源。

辅助记忆

1. 负载均衡

2. 修改线程池配置

3. 临界值的确认

4. 限流方案以及好处

5. 批量持久化到数据库和定期清理无用连接

怎么设计一个私聊的拉黑功能？

建立一张表，表的列有拉黑人 id 和被拉黑人 id，用户 A 拉黑用户 B 则向表中插入一行拉黑人 id 为用户 Aid，被拉黑人 id 为用户 Bid 的记录，用户 B 向用户 A 发送私聊消息前先查询

拉黑表中是否有被拉黑用户 id 为用户 Bid，拉黑用户 id 为用户 Aid 的记录，若有则发送失败，用户 A 取消拉黑用户 B 则是删除表的对应记录。

用户上线后是怎么收到离线消息的消息通知的？

上线后会查询消息表里状态为未读的消息数量，若不为 0 则前端会渲染出带未读消息数的红色数字，用户看到后点击图标可以跳转到私聊页查看未读消息。

单个实例比如实例断了重启了，之前服务端的旧消息还没转发给另一个客户端，重启后需要转发给该客户端的新消息又来了，新旧消息怎么保证有序性？

将消息缓存到 redis 并加上时间戳，要转发新消息给某客户端前先查看 redis 中是否还有该客户端没转发的消息，有的话则先按时间戳顺序转发 redis 中的消息给客户端再转发新消息。

WebSocket 吃性能有没有解决办法？

WebSocket 本身是一种高效的全双工通信协议，但在高并发场景下确实可能出现性能问题，主要体现在连接管理、数据传输效率和资源占用上。解决这个问题可以从几个方向入手。首先是连接优化，比如使用连接池管理长连接，避免频繁创建和销毁连接带来的开销；同时合理设置心跳机制，及时清理无效连接，防止资源浪费。

其次是数据层面，尽量压缩传输的数据体积，比如用 gzip 或专门的二进制协议替代文本格式，减少带宽和解析成本；另外避免不必要的消息推送，只传输关键信息，降低处理压力。再者是服务端架构设计，采用分布式部署分散压力，结合负载均衡将连接均匀分配到不同节点；还可以引入消息队列缓冲突发消息，避免瞬间高并发压垮服务。

如果同一个用户，他拿两个浏览器打开你这个私聊页面，你这个 websocket 对象是几个？

WebSocket 对象会有两个，因为建立一次 websocket 连接就会有一个，但是由于有初始化步骤，map 存储的 websocket 对象会修改一次，后边的连接对应的 websocket 对象覆盖前边的，这样就会出现一个问题，其他人消息想发给当前用户时，只有后边的连接对应的页面能收到实时消息，因为 map 里取 websocket 对象时只能取到后边页面的连接的 websocket，要解决这个问题可以修改 map 的结构，值改成集合，然后在多个浏览器都进入私聊页面时把每个浏览器的连接的 websocket 对象都存到集合里，转发消息时每个 websocket 对象都取出来转发一遍，或者做单设备登录的限制。

（单设备登录限制怎么做参考下边的内容）

统一授权、鉴权和单点登录

说说你项目里的统一授权鉴权和单点登录是怎么做的？

项目由六个服务组成，所有请求都统一进入网关服务进行鉴权
若请求路径在放行路径内则直接放行，路由转向实际业务服务
若不在放行路径则检验是否有合法且未过期的 token

有则放行并将请求路由转向实际业务服务

无则返回 401 状态码提示前端该用户需要登录

同时网关服务负责登录注册，登录成功后授权，返回有效 token 给前端用于后续请求的鉴权这样就避免了在每个业务服务重复编写鉴权授权功能代码

同时因为请求先在网关鉴权再转向其他服务，只需在网关服务登录过一次拿到 token 即可在访问其他服务时都保持登录状态，实现单点登录。

单点鉴权为什么使用 spring security+jwt 技术？

Spring Security 是成熟的安全框架，提供了完整的认证和授权机制，包括用户身份校验、权限控制等核心功能，能快速搭建安全防护体系，无需从零开发基础安全逻辑。

而 JWT 作为一种轻量级的令牌机制，适合在分布式系统中传递用户身份信息。通过数字签名保证信息的完整性和真实性，且令牌本身包含用户身份和权限等关键信息，服务端无需存储会话状态，解决了传统会话机制在分布式环境下的共享难题，减少了服务端存储压力。

两者结合时，Spring Security 负责处理认证流程，认证通过后生成 JWT 令牌返回给客户端；客户端后续请求携带 JWT，Spring Security 通过解析令牌完成身份验证和权限校验，既利用了 Spring Security 的安全管控能力，又借助 JWT 实现了无状态的分布式鉴权，兼顾了安全性和系统扩展性。

双 token 实现无感刷新

你简历中提到双 JWT 实现无感刷新 token，说说这个流程？

1. 项目中原先采用的是 token 鉴权，jwt 作为具体 token 实现形式，而 jwt 会有过期时间，设置的过长被盗用后造成的影响会更大，但设置的短又会出现用户进入网站一段时间后 token 就过期了，过期后的访问鉴权不通过会通知用户重新登录，影响用户体验

2. 因此选择单 token 进行无感刷新，设置过期时间为 30min，前端每 25min 发送一次携带有效 token 的请求，获取新的过期时间为 30min 的 token，这样在用户进入网站并登录后就可以一直保持登录态，且前端会在每次打开网页时发送一次刷新 token 请求，以此做到即使关掉了网站的所有页面，只要在 30min 内重新打开网站即可续上 token

3. 但因为刷新 token 接口为了防止被恶意盗刷需要请求中存在有效 token 才能执行成功，还是会出现超过一定时间比如两天后 token 过期，再打开网站后的刷新 token 请求失败，登录态失效的问题，因此做了长短 token，30min 过期的短 token 只用于鉴权，7 天的长 token 用于刷新 token 请求时的凭证，每次刷新请求只要存在有效的长 token 即可通过，返回新的 30min 过期短 token，同时考虑到若长 token 过期时间不更新则会出现过了即使频繁进入网站，从登录时刻开始过了七天后仍然会有登录态失效的问题，因此选择在刷新短 token 同时刷新长 token，以此做到只要七天内登录过一次即可无感刷新 token。

4. 我觉得还可以用单 token 滑动+过期的形式，将用户 token 存到 redis 中，设置缓存七天过期，每次用户请求到后端时都会检验，若有对应 token 的缓存则说明 token 未过期将 token 的对应缓存的过期时间设置到请求时间的七天后并放行请求，若无对应 token 的缓存则说明 token 已过期，提示前端重新登录，这样也可以实现无感刷新 token

且可以少存储一个长 token，复杂度降低，但相比双 token 流程短 token 用作鉴权的凭证长 token 用作刷新 token 的凭证，单 token 把鉴权和刷新权限耦合到一起了，没有进行很明确的职责拆分。

辅助记忆

- 1.最初不自动刷新方案
- 2.单 token 刷新方案
- 3.双 token 刷新方案
- 4.优化方案和优缺点

你说前端每 25 分钟发送一次刷新 token 请求，如果同时发起多个刷新请求（如网络延迟导致请求堆积），如何避免短时间内重复刷新（比如 1 分钟刷新几百次）？

每次发送请求前先查询 redis 中是否有刷新 token 请求对应的键值对，没有的话则发送一次请求，并且 redis 中设置个 1 分钟过期的值类型键值对，如果有的话就请求失败，通过这样将请求频率限制在最高 1 分钟 1 次。

Jwt 的缺点是什么？

无法主动撤销，在有效期内始终有效，由于不在服务端存储，只校验 jwt 令牌的有效性和过期时间，因此退出登录或修改密码后持旧的 jwt 仍旧能通过鉴权，以及客户端存储容易被窃取。

Jwt 的签名有存服务端吗？

没有，服务端只校验 jwt 是否合法以及是否过期，不会存 jwt 的签名。

若刷新 token 被泄露，可能会带来哪些安全风险，你在项目中采取了什么措施来防范这些风险？

- 1.被恶意网站或者用户盗刷 token 的风险。
 - 2.一个是添加设备指纹，给每个登录的用户返回一个存在 http-only-cookie 里的，用 uuid 来取值的设备指纹，刷新长 token 的请求会额外校验用户的设备指纹是否存在且合法，若不满足条件则说明是被窃取了长 token 来发送的请求，当次请求不会通过，这样盗刷 token 需要将长 token 和设备指纹一起盗取才能成功，增大了盗取难度。
 - 3.一个是可以在返回前端长 token 时在长 token 中存一个 userid，用户每次访问接口时根据用户的真实 ip 调用 api 获取到用户的所在省市并保存到 redis 中，userid 做键，真实省市做值，每次长 token 的刷新请求都会从 redis 中查询该用户的省市，再根据这次请求的 ip 地址去调 api 获取这次请求的发起者所在省市，若值不匹配则说明被窃取了，刷新请求会失败。
- 同时根据 ip 做校验的方式如果用户使用了代理登录网站会出现误伤的情况，但这种情况出现概率小，而根据 ip 做校验确实是有效加强长 token 安全性的方法，如果权衡之下安全性更重要就可以采用。

辅助记忆

1. 有哪些风险
2. 添加设备指纹和设备指纹赋值，存哪里，怎么用
3. 记录每次请求的真实 ip 以及什么情况适合用

设备指纹存在哪？多久过期？

http-only-cookie。可以仿照无感刷新流程，设置为七天过期，每次有请求都返回一个新的七天过期的设备指纹给前端。

长短 token 存储在那里的？

http-only-cookie 里。

在实际应用中，双 JWT 无感刷新 token 的机制是否会受到服务器时间同步问题的影响？如果受到影响，你是如何解决的？

会。服务端使用获取时区时间 api 拿到时区的时间避开服务器时间的影响。

如果有多个实例，用户在一个实例上登录了，另一个实例上也登陆然后做了一些修改，怎么让旧的 token 失效？

可以在服务端存储某个用户对应的 token，在其他实例上登录的话就修改该用户的 token，同时鉴权步骤也加上校验 redis 中用户对应 token 的步骤，就能实现其他实例登录后旧 token 失效的效果。

如果我同时打开两个你的 b 站项目的网站，token 是一样的吗？

如果同时打开了两个网站，在打开的一瞬间会携带长 token 发请求向后端获取新的长短 token，后端将长短 token 设置为 http-only-cookie 并返回，浏览器接收响应后自动存到 http-only-cookie 里，且由于现实中各种影响因素比如网络延迟，现实中时间戳只有接近相同不可能完全相同，因此后边响应的 token 会覆盖前边响应的 token。

大模型调用和多账号凭证

你简历中提到集成了讯飞星火大模型，说说怎么做的？

1. 原先大模型调用是发送提问后能获取到文本、图片、ppt 的回答。
2. 获取文本答案实现形式是用户输入提问，将提问和用户的 id 作为内容发送 websocket 消息给服务端

服务端查询 map 中是否有和该用户绑定的大模型处理类对象，有则直接调用该大模型处理类对象的方法

没有则创建一个和该用户绑定的大模型处理类对象后调用方法然后再将该新建对象放入 map

调用的方法会新建和大模型方的 websocket 连接，将提问内容存入该用户的历史问答集合，然后连带之前的问答历史记录作为 websocket 消息内容一起发送给大模型方

接收大模型方的 websocket 消息形式的回答后通过 userid 找到用户对应的 websocket 对象传递回客户端，然后将回答放入历史问答集合中

3. 图片答案和 ppt 则是客户端发送 http 请求到服务端然后服务端通过发送 http 请求的形式获取大模型方响应并转发回客户端

虽然后端与讯飞星火方的文生文功能交互被接口限制只能是 websocket 形式，但前后端交互在不采用 websocket 的前提下也可以采用 http 形式，然后前端建立和后端的 sse 连接，后端通知前端用 sse 做单向推送。

辅助记忆

- 1.集成后有什么功能
- 2.文本回答怎么实现
- 3.图片、ppt 回答怎么实现

多账号凭证突破 QPS 为 2 的限制是如何做的？

1.每个账号可以申请一个有免费额度的 key，而这个 key 对应文本、图片、ppt 功能都各自有 QPS 为 2 的限制，也就是一个账号最多只能有 2 个人同时使用文生文或文生图、文生 ppt 功能，但各个功能间互不影响，彼此的使用人数互相隔离

所以若在 2 个人已经使用了文生文或文生图、文生 ppt 功能且都未使用完毕的情况下第 3 个人使用该功能就会陷入阻塞状态直到前面 2 个人中有一个已经使用完毕

2.为了避免出现这种影响用户体验的情况于是用多个账号去扩充 QPS 上限，每个账号的每个功能都记录当前使用人数，当用户需要用到其中一个功能时则去账号集群中找该功能使用人数小于 2 的账号

使用该账号并将该账号的该功能使用人数加一，使用完毕后再将该账号的该功能使用人数减一

类似 redis 分布式锁的实现，以这种方式实现突破每个功能的 QPS 上限，若有两个账号则可以有 4 个人同时使用文生文或文生图、文生 ppt 功能，账号越多每个功能的 QPS 上限越大。

辅助记忆

- 1.qps 为 2 的限制是什么意思
- 2.如何突破限制
- 3.突破方案的参考文献和突破效果

大模型流式输出是怎么实现的？

讯飞星火的接口就是流式的返回，因此沿用就可以实现流式输出。如果要自己手动实现可以在后端把所有的内容接收后定时返回一部分，让前端配合做好样式渲染出流式输出的效果。

在通过存储多个账号凭证并在使用时查询使用后修改凭证状态的方式突破讯飞星火 QPS 为 2 的限制时，如何保证凭证管理的安全性和准确性？

目前项目里凭证是存在后端程序里写死的

因此被攻破的概率不大，但如果凭证是存在 redis 中或文件中则可以进行加密存储

从存储的位置取出来拿到 java 程序中用时，java 在进行一层解密，以此来保证凭证的安全性。

准确性则是定期使用凭证，如果用某个凭证和大模型进行交互失败则说明凭证无法使用了，需要及时清理来确保凭证的准确性。

在高并发场景下，多个用户同时请求大模型服务，如何优化账号凭证的分配策略，确保每个用户都能尽快获得服务？

创建定时任务定期查询并得出可用列表，这样每次用户要使用时直接从可用列表里拿即可。

假如说用户对生成的图不满意，需要调整，项目里能做到在原图上的连续的调整吗？

使用的讯飞星火 api 不支持传图片过去生图，如果要做到在原图上的连续的调整我的思路是换一个可以接受文本图片一起传过去的模型的生图 api，比如豆包提供的 api，然后前端提问

到后端，后端调用这个 api 去进行连续的调整。

怎么保证用户获取凭证的线程安全的？

可以通过加锁的形式保证，账号集群中的每个账号都是一个自定义对象，对象的属性有使用人数 count，连接大模型的密钥 key，功能的枚举，这个枚举的值有文生文，文生图，文生 ppt，然后将账号集群以 list 数据类型的形式存到 redis 中。在原先的遍历过程中加锁，比如调用文生文功能，那就在这个账号集群里找到枚举值为文生文的一批对象，然后访问这批对象中的每个对象的时候都加锁，确保同一时间只有一个线程能访问这个对象，这个线程查询了对象的当前使用人数并根据当前使用人数决定是否用这个对象的 key，做完这些后释放锁给下一个线程。

如果大模型功能调用过程中凭证过期了怎么办？

如果是图片或 ppt 功能这样一次性返回前端所有内容的，就切换其他可用凭证先在后端进行重试，若无其他可用凭证或其他可用凭证重试过程也失败了则响应通知前端请求失败，请稍后重试

如果是文本这样流式返回前端内容的则在后端返回响应给某个客户端前，先记录所有讯飞星火那边已经返回后端的响应内容，重试时在前端提问后面加上以及这是一部分回答请紧接这部分回答继续

确保呈现给前端的上下文连贯下去，若无其他可用凭证或其他可用凭证重试过程也失败了则响应通知前端请求失败，请稍后重试，并让前端将之前不完整的回答全清除。

如果所有的账号都被占满了怎么办？除了新增账号

可以先阻塞后端线程，并定时轮询，直到有一个可用的账号了再解除线程的阻塞，或者每次阻塞线程后将阻塞的线程加入队列中，同时使用账号的线程使用完毕后选择一个被阻塞的线程解除阻塞，并将账号传递过去。

怎么选择的被阻塞的账号？

可以随机选择，也可以将这些被阻塞的线程包装到一个带有时间戳的自定义对象中，选择时按时间排序，选择开始阻塞时间和当前时间间隔最久的。

说一说你是怎么设计 prompt 的？

项目中没有刻意用到 prompt，平时提问 ai 时会有用到，一般是加一些条件或假设，让 ai 的回答更专业更精确，比如假设是一名技术专家，要求回答时带有详细解释这种 prompt。

讯飞和豆包哪个的生成质量更高？

豆包，讯飞只是免费额度多，但实际生成质量不如豆包。

这个账号池是用 Redis 来维护的是吧？那你用账号池存储它的调用次数、并发状态的话，如果是你的 Redis 宕机了，你要怎么办？

对于 Redis 宕机的情况，从多个层面保障账号池的可用性。首先，采用 Redis 主从复制架构，主节点负责读写操作，从节点实时同步数据，一旦主节点宕机，可快速切换到从节点继

续提供服务，减少数据丢失风险。

然后定期对 Redis 数据进行持久化，通过 RDB 快照或 AOF 日志的方式将数据备份到磁盘。当 Redis 完全不可用时，能基于最新备份快速恢复账号池数据，包括各账号的调用次数、并发状态等关键信息。

另外，系统会加入 Redis 健康检测机制，一旦发现 Redis 异常，会暂时将账号池数据缓存在本地内存中作为过渡，并触发告警机制及时运维人员处理。待 Redis 恢复后，再将本地缓存的数据同步回 Redis，确保账号池功能不中断，避免影响大模型的正常调用。

通过这种多机制结合的方式，既能 Redis 宕机带来的影响降到最低，保障整个集成流程的稳定性。

数据同步

请你讲讲数据同步的流程？

1. mysql 到 es 数据同步是把 mysql 的所有视频表、用户表的数据同步到 es 的视频索引和用户索引中。
2. 每次对视频表、用户表的记录有影响的增删改操作执行时会远程调用消息队列的生产消息接口往同步消息的主题中推送增量消息，并在监听该主题的消费者中将增量消息缓存在 redis
3. 使用 xxl-job 每十五分钟执行一次定时任务，该定时任务先查询 redis 中标识用户索引全量同步过的键值对是否存在，若不存在则说明未进行过全量同步
4. 先从 mysql 中查询用户表的所有数据，将所有数据导入 es 中，再向 redis 中存一个标识用户索引全量同步过的键值对，后续定时任务执行时就能查询到该键值对存在从而不再重复全量同步，视频索引也是同样步骤。
5. 全量同步逻辑后进行增量同步处理，从 redis 中取出对视频、用户表的增删改操作列表按先新增再删除再修改的顺序批量同步，同步时将单个的新增、删除、修改对象添加到批量处理对象中进行批量操作减少和 es 的交互，减轻 es 的负担，再递归的进行对 es 的批量操作
6. 每次批量操作时收集失败的单次新增或修改或删除操作，添加到一个新的批量处理对象中再去进行一次递归，直到不再有失败的单次操作，或者递归次数超过十次，这样最大程度防止失败，也能避免因特殊原因如 es 挂掉等导致一直失败，程序一直递归造成堆栈溢出问题。
7. 若超过十次还有失败的单次操作则将该失败操作记录到文件中且发送邮箱将该情况通知出去。
8. 同时初始化存放所有文档 id 的 hashset，先新增同步再删除同步并对 hashset 的元素进行同样的新增删除，在将修改操作的单个操作对象添加到批量操作对象时，对每个修改操作对象都查询 hashset，若判定该修改操作对象对应的文档 id 不存在 hashset 则说明这个文档也就是对应的那一行 mysql 记录的最终状态是被删除，那么就无需进行修改操作，也就无需将该修改操作对象添加到批量操作对象中，减少对 es 的操作，减轻对 es 的负担。
9. 但这样的话数据一致性不够强，会有延迟同步的情况，在对数据一致性要求高的业务场景下可以采用双写，在更新 mysql 的同时也更新 es 来做到强一致性，但这样做对服务器的负担会更重。或者每次定时任务执行时都直接查 mysql 数据库进行全量更新，这样实现简单，但表数据大的情况下会对数据库造成较大压力，或者引入 canal 实现，但引入 canal 需要额外部署中间件和增加新的依赖，复杂度变高了，而且自己实现有利于加深对技术的理解

辅助记忆

- 1.实现效果
- 2.增量消息缓存
- 3.全量同步判断
- 4.全量同步执行
- 5.增量同步执行
- 6.增量同步递归
- 7.递归防报错处理
- 8。为什么用 `hashset`
- 9.双写、全量更新、`canal` 优化方案和优缺点对比

随着数据量的不断增加,这种数据同步机制的性能是否会受到影响?你有什么优化思路来应对数据量增长带来的挑战?

全量只执行一次,因此优化主要在增量,可以把定时任务执行间隔时间变长来确保每次执行时处理的增量消息更多,可以采用多线程来并行对视频和用户的数据同步提升效率,同时还可以考虑提升服务器配置来应对数据量增长情况下性能变低的问题。

hashSet 什么时候初始化的?

定时任务启动时, 查询出 `es` 中视频和用户索引的所有文档 `id` 并存入对应的 `hashset`。

HashSet 的初始化步骤是否可以减少查询 `es` 的次数?

可以在搜索服务启动时初始化 `Hashset`, 这样只有每次重启搜索服务时才需要重新初始化。

若 `Redis` 中的增量消息丢失(如 `Redis` 宕机), 如何恢复未同步的数据?

先通过预先配置好的持久化方式恢复 `redis` 中的数据, 再等待下一次同步, 把未同步的数据同步上去。

如何保证增量同步操作的幂等性? 例如, 重复消费“新增”消息是否导致数据重复?

新增之前先查询是否该 `id`, 有的话就不新增

数据同步期间搜索服务还能用吗?

能。

换成 `canal` 怎么进行数据同步?

部署并启动 `MySQL`、`es`、`Canal` 服务端与客户端。且开启 `mysql` 的 `binlog`, 且格式设为 `ROW` 模式。然后配置 `Canal` 服务端, 在服务端配置中指定 `MySQL` 的连接信息、监听的数据库和表, 让 `Canal` 模拟 `MySQL` 从库, 订阅 `MySQL` 的 `binlog` 日志。

再配置 `Canal` 客户端, 客户端关联 `Canal` 服务端地址, 同时配置 `Elasticsearch` 的连接信息, 并定义 `MySQL` 表字段与 `ES` 索引字段的映射关系。

接着 `Canal` 服务端实时读取 `MySQL` 的 `binlog`, 解析出增删改等数据变更事件, 将变更数据封装成统一格式。

最后 `Canal` 客户端从服务端获取变更数据, 按照预设的字段映射规则, 将数据转换为 `ES` 可识别的文档格式, 再通过 `ES` 的 `API` 将数据写入或更新到目标 `ES` 索引中, 完成同步。

如果程序中有多个定时任务，某个定时任务（本身）出错，怎么解决？

根据定时任务所在的完整链路进行问题的定位，定位后修改有误代码，同时修正错误代码已经造成的对数据的影响，从日志中找到定时任务执行的原始数据，将错误数据修正成原始数据在正确代码逻辑下的最终正确数据。

如果 Redis 宕机了，怎么知道 es 和 mysql 有哪些数据没有同步？

可以采取对账的方式，先查 mysql 的所有数据，再查 es 的所有数据去做对比得出哪些更新丢失了，也可以给 Redis 中存储的所有增量消息再加一层保障措施，再记录一份增量消息到服务器文件里，Redis 出问题了就对照文件，同时文件也可以复制多份，放到多个服务器，防止只存一个服务器而某个服务器又宕机，以及还可以加心跳机制监控，定期读取文件确保服务器正常运行，有出问题及时通知开发人员

还可以 mysql 建同步任务表，表存储增量消息和任务是否完成的字段，这样还能顺便利用 binlog 的数据复原防止 mysql 中存储的增量消息也丢失。

如果是在集群模式下多台服务器执行同一个任务，怎么保证任务执行的不重复性？

1.可以在 mysql 数据库中设计一张任务执行记录表，包含任务名，任务计划触发时间等字段并为这些字段创建唯一索引。当任务被触发时，尝试向该表插入一条记录。只有第一个成功插入的实例能继续执行，其他实例因为违反唯一约束而插入失败，从而知道自己不需要执行。任务执行完毕后，可以删除该记录，或者保留作为日志，但需要清理旧记录，优点:是实现相对简单，可利用现有数据库，缺点是 对数据库有一定压力。

2.还可以选择用 Redis 为当前任务实例生成一个唯一的 key，所有触发的实例 set 命令尝试执行设置一个 30 秒的锁，SET 命令返回成功的实例获得了锁，可以执行任务。

其他实例收到失败响应，知道已有实例在执行，直接退出，获得锁的实例执行任务。

任务执行完毕后，主动删除 key，可以使用 Lua 脚本保证原子性：先判断值是否是自己设置的，再删除。即使忘记删除或中途宕机，锁也会在过期时间后自动释放。

优点是性能高，比数据库锁通常更快。有现成的原子命令和过期机制。

缺点是 Redis 的可用性和一致性需要考虑，特别是在主从切换或哨兵/集群模式下，可能存在锁丢失或误锁的风险。需要合理设置过期时间。

辅助记忆

1.数据库记录去重实现和优缺点

2.redis 记录去重实现和优缺点

Mysql 和 es 同步的数据有哪些？

用户表、视频表的数据。

为什么用户的数据也要同步到 es？

因为有做聚合搜索功能，用户和视频都能进行搜索，因此用户数据也需要同步。

数据同步的意义？

因为项目用到了 es，如果不做数据同步 mysql 和 es 的数据就会不一致，那么走 es 搜索出来的数据就和 mysql 不一致。

定时任务是通过扫描整个表来确定有没有全量同步过吗？

是通过全量同步后在 `redis` 中存键值对来确定有没有全量同步过的。但也可以搜索 `es` 中用户索引和视频索引，若能查出数据则说明进行过全量同步。

你这里项目的 `rocketMQ` 在这个流程中起到的是什么作用？

统一处理增量消息缓存到 `redis` 的工作，并且利用 `rocketmq` 本身的重试机制尽量改善缓存增量消息失败的情况。

你的定时任务有没有防止出现错误的机制？

1.有采用多种机制来防止、检测和处理错误，以提高其健壮性和可靠性

单纯地“防止”所有错误是不可能的，但可以通过设计和实施良好的策略来最小化错误发生的概率，并在错误发生时有效地处理它们

在任务执行的核心逻辑代码块中使用 `try...catch` 捕获预期和意外的异常，并记录详细错误信息，当捕获到错误时，记录详细的日志，包括错误类型、错误消息、堆栈跟踪、发生时间、相关参数、任务的启动、关键步骤、完成状态以及执行耗时等，可以在出现问题时进行更好的对比和排查

2.还可以给任务设置一个合理的执行超时时间，如果任务执行时间超过阈值，可以强制终止它，防止单个任务卡死影响其他任务或耗尽系统资源

对于一些瞬时性错误如网络抖动、数据库连接暂时失败，可以配置自动重试策略，例如，延迟几秒/几分钟后重试，最多重试 `N` 次，在重试时逐渐增加等待时间，避免对下游系统造成持续压力

3.还有需要确保任务执行多次和执行一次产生的效果是相同的。这对于防止因重试或任务调度重叠导致的数据重复处理或状态错误很重要。

辅助记忆

1.声明只能最小化错误概率以及记录日志方案

2.给任务设置超时时间和重试策略

3.确保幂等性

失败处理那里，假如一直失败，会不会导致数据不一致，你是怎么解决的？

可以在 `redis` 中用哈希数据类型存失败的同步操作，键为文档 `id`，值为该文档失败的新增修改删除操作列表，以此来聚合对同一个文档的失败操作，且列表的元素只需要标识是新增还是修改还是删除操作，创建定时任务定期取出文档的失败同步操作，按删除新增修改的顺序进行判断，如果有删除同步则说明这条数据最终状态是被删除，查询 `es` 该文档是否存在，不存在的话这个文档的所有失败同步操作都不需要执行了，存在则重试删除同步，然后跳过新增和修改的同步重试。如果无删除有新增则直接根据文档 `id` 查询 `mysql` 中这行记录的最终状态并同步到 `es`，且直接省去修改同步的重试。如果只有修改则类似新增，根据文档 `id` 查询 `mysql` 中这行记录最终状态并同步到 `es`。

为什么要用 `redis` 缓存增量消息？

因为批量处理就需要有个位置先缓存增量消息，执行批量处理时才能从这个位置取增量消息，而 `redis` 有完善的数据持久化机制，即使宕机了也可以靠 `aof` 或 `rdb` 文件恢复数据，丢失增量消息的概率低。

业务代码中生产消息到消息队列这一步失败怎么办？

如果要求强一致性就回滚 `mysql` 的修改，如果不要求强一致性则先不动 `mysql` 的修改，并且

Es 的分词器用的哪个？

没有特别设置分词器，用的默认的，不过对中文分词器有印象，能改善 `es` 默认的分词器中文分词能力较差的问题，比如 `IK` 分词器，支持细粒度和粗粒度，可以自定义词典。

如果通知人工不是最优的，现在要在系统内处理，保证一致性更高，你要怎么做呢？

如果一直失败的话且需要保证一致性足够高的话可以将 `mysql` 中对应的增量回滚来保证一致性足够高，这样的话需要针对不同类型的增量额外记录一份信息，新增类型的增量回滚，需要额外记录新增的 `mysql` 记录的 `id` 用于删除，删除和修改类型的增量回滚则需要保存被删除前或被修改前的记录具体内容来增加被删除了的数据或复原被修改了的数据。

同步为什么要用 redis 缓存增量消息不直接从 MQ 消费同步？

因为直接从 `mq` 消费同步实现比较简单，想自己写一些其他的实现加深对技术的理解。而且当时还想过另外一个同步的方案，`Redis` 中维护一个键为操作类型，值为增量集合的哈希类型键值对，设定批处理阈值比如 50，每次收集到增删改增量消息时查看 `Redis` 中所有键对应的集合的元素数量总和是否到达 50，没到达则将增删改增量消息存入对应操作类型的键值对中，到达了则进行增量同步，这样相比原流程定时任务，有可能出现一段时间都没有增量消息，导致一直没到达阈值，增量消息一直没同步的问题，因此还需要添加超时机制，若超过一定时间一直没进行过同步就强制同步。

搜索部分是直接调 es 吗？

是的，在搜索服务利用 `es` 的 `multimatchquery` 查询 `api` 编写接口，并在视频服务中远程调用搜索服务的 `api`。

数据同步批量修改如何保证顺序性？

1.在完整流程中能想到的批量修改乱序有两个情况

一个是在 `mq` 中消费增量同步消息时，对同一个视频或用户的多个修改的消息出现了乱序，修改时间靠后的修改操作对应的增量消息反而被先消费存到 `Redis` 中，导致在 `Redis` 存储修改增量的列表中时间戳靠后的修改操作的顺序反而靠前

那么假设一个视频被修改了两次，同步时数据的最终状态就不是最后一次修改而是第一次修改，这样就会出现乱序

2.还有一个是同步时会有对失败的增量同步进行递归重试的步骤，如果消费增量同步消息无误，同一个视频或用户的多次修改操作也有可能时间戳靠前的修改操作失败了，但时间戳靠后的修改成功了，然后重试时间戳靠前的修改操作，导致数据的最终状态并不和 `mysql` 中一致

2.这两种情况都可以通过修改缓存到 `Redis` 的修改增量的结构来保证顺序性，向修改增量的结构中添加一个时间戳，取修改增量进行批量修改时按用户或视频进行聚合，同一个用户或视频的修改增量按时间戳排序进行数据同步

4.让对同一个用户或视频的多个修改操作达成原子性，要么全部成功要么全部失败，防止出现脏数据，这种方案可以保证数据同步批量修改的顺序性，但和 `es` 的交互更多，对 `es` 的负担更大，需要权衡顺序性和服务器减轻负担的优先级。

辅助记忆

- 1.出现乱序的情况一，redis 里存储的增量顺序乱了
- 2.乱序情况二，重试同步过程导致乱序
- 3.聚合同一个视频或用户的增量消息来批量消费确保不乱序
- 4.达成的效果和优缺点

为什么不直接把数据存到 es，而是要存 mysql 里？

Mysql 有相比 es 更加完善的事务机制来保持数据操作的一致性，以及更强大的数据备份和恢复能力，因此需要在 mysql 中也存一份数据。

数据同步的时候如果 es 挂掉一段时间之后重启，会重新全量同步吗？

不会，因为全量同步只进行一次。

为什么要设计数据同步和合并请求？

平时学习时比较追求方案的升级和优雅的实现，因此设计数据同步和合并请求这两个流程。

Es 的索引是怎么建立的？

参考视频表和用户表的列，每个字段复刻，建立的视频索引和用户索引。

如果是你的 ES 宕机了，你搜索该怎么办？

可以做功能切换或降级，临时将搜索请求路由到 MySQL，暂时承接搜索流量，当流量过大时则需要做功能降级提示，向用户友好反馈当前搜索服务临时维护，建议稍后尝试。

以及部署集群，当一个节点宕机后马上切换另一个节点接管服务。

服务间的为什么要用 openfeign 而不用其他 rpc？

主要在于它能更好地适配基于 HTTP 的服务通信场景，且与 Spring 生态深度融合，开发体验更流畅。

OpenFeign 本质是对 HTTP 客户端的封装，基于 RESTful 风格设计，通过注解声明式定义接口，无需手动编写 HTTP 请求代码，大大简化了服务间调用的开发成本。对于使用 Spring Cloud 技术栈的项目来说，它可以无缝集成，配置简单，比如通过 @FeignClient 注解就能快速定义服务调用接口，自动处理负载均衡、服务发现等问题，无需额外引入复杂的注册中心或协议适配逻辑。

相比之下，很多传统 RPC 框架如 Dubbo 基于私有协议，虽然在性能上可能略优，但需要额外维护服务注册中心的适配、协议序列化等细节，且在跨语言、跨平台兼容性上不如 HTTP 协议通用。如果项目本身依赖 Spring 生态，且服务间通信更注重开发效率和协议通用性，OpenFeign 这种轻量级、声明式的方案会更合适。

生产消息到消息队列失败怎么办？

重试，如果重试多次还是失败则回滚 mysql。

回滚 mysql 时有其他事务在更新数据怎么办？

回滚一行记录时用写锁这样的行锁，比如 select for update，阻塞其他事务读取当前行。

MYSQL 的数据量是多少，需要使用到 ES 来辅助查询？

没有特地造数据测试，但是有大概查过范围，数量级到百万时 mysql 有合适索引的查询速度为 10-100 毫秒，无合适索引或者索引失效的情况大概在几秒到十几秒，而 es 在百万级数据情况下查询速度通常在 10-500ms，因此到百万级数据时就可以使用 es 辅助查询了。

合并请求

请你讲讲合并请求的流程？

1. 合并请求是在某些请求的流量低于阈值时保持原先请求路径，在流量高于阈值时转向新请求路径，且该路径对应接口做了合并统一处理然后再统一返回的方式，以此来减少对数据库的压力，加快高并发环境下的响应速度。
2. 实现方式是程序启动后执行定时任务，初始化有可能合并请求的接口的 qps 值为 0，后续每 1s 重置一次 qps 值然后请求进入网关后判断请求路径是否是需要考虑合并请求的路径，是的话查询该请求的 qps 值是否大于等于临界值，大于等于就将请求路径转成合并请求路径，否则保持原先的单独查询不变，再将该请求的 qps 值加一
3. 进入合并请求接口会创建一个自定义请求对象，属性包含唯一的请求 id、用户 id 和一个 completablefuture 对象，再将该对象放入 Linkedblockingqueue 队列中，然后调用 completablefuture 的 get 方法阻塞当前线程，同时在 Springboot 程序启动后会初始化一个每隔 500ms 执行一次的定时任务，每次定时任务执行时会将缓存在队列中的自定义请求对象取出来，根据这批自定义请求对象的用户 id 组成的集合进行批量查询，查询结果转换成请求 id 为键用户信息为值的形式，再遍历之前取出来的自定义请求对象集合，通过调用 completablefuture 的 complete 方法，将对应每个自定义请求对象的请求 id 的用户信息填充到 completablefuture 对象中，然后该次请求对应被阻塞的线程也会被唤醒并返回用户信息给前端。
4. 但这样有一个缺陷，completablefuture 常用的 get 方法没有超时机制，因此若过长时间没有拿到查询结果就会一直阻塞
5. 因此优化方案是返回前端的形式由指定 completablefuture 类型为用户对象类型再调用 get 方法等待用户对象被填充后返回前端，换成指定 Linkedblockingqueue 类型为用户类型，再监听 Linkedblockingqueue 中的数据，原先的自定义请求对象的 completablefuture 换成 Linkedblockingqueue，查询后对每个自定义请求对象也就是每一次的请求都进行一次填充用户对象到 completablefuture 的操作也换成对每个自定义请求对象都进行一次填充用户对象到 Linkedblockingqueue 中，一旦自定义请求对象中的队列有数据就代表查询结果已经被放入，返回前端用户信息，设置队列超时时间为 3s，超过三秒某次请求对应的队列中还未数据则会直接放弃阻塞线程，返回 null 给前端。也可以用 get 方法的可传入超时时间的重载方法，等待时间超过该方法的传参值后会抛出异常，捕获该异常并返回 null 给前端。
6. 同时不全部走合并请求的原因是合并请求的查询定时任务被设置为 500ms，防止设置的过短对数据库压力大，那么如果低并发的情况单次查询速度可能比合并请求后批量查询速度更快，而且低并发对数据库压力也不算大，不用考虑为了降低数据库压力而合并请求的情况，也就没必要合并请求。
7. 临界值的确定可以启动后端所有会用到的程序后用测试工具比如 jmeter 指定请求路径和 qps 值后发送请求，得到这个 qps 下全部走合并请求的平均响应时间，看多少 qps 时单独请求的平均响应时间大于合并请求的平均响应时间，就把这个值定为 qps 的临界值。

比如从 qps 为 1 开始，逐步递增 1，观察到达多少时合并请求比单独请求更快，这样很稳，但也很慢，所以可以改良成二分式查找，因为 qps 区间最左端是单独请求速度大于合并请求速度，区间最右端的是单独请求速度小于合并请求速度，找到了第一个单独请求速度小于合并请求速度的端点 A，那就把区间再缩小，从左端点到这个端点，直到区间缩小为左端点等于右端点。

辅助记忆

- 1.合并请求是做什么
- 2.请求是否需要走合并请求逻辑
- 3.使用 `completablefuture` 的形式合并请求如何做
- 4.`completablefuture` 方式的缺陷
- 5.换成 `linkedblockingqueue` 和 `get` 重载方法怎么做
- 6.为什么有时单独请求有时走合并请求
- 7.临界值确定方法

平均响应时间是怎么测的？

多次测量，取多次响应时间得到平均值。

视频上传的时候，断点上传的部分，怎么判断哪些已经上传过了。没有上传的那部分，怎么判断有没有另一个线程正在上传呢？

合并请求中也有涉及异步，为什么不采用消息队列来完成这个操作呢？

也可以采用消息队列，不过消息队列默认是单个消息的处理，而项目中异步执行的是对请求的批量处理，如果要用消息队列就需要往 `java` 内存或者 `redis` 中存自定义请求对象，当达到一定数量后取出来批量处理。（参考数据同步那个直接用消息队列进行数据同步的问题，思路是一样的）

统计 qps 还有别的方法吗？

可以用 `sentinel` 进行 qps 统计，好处是有现成 api 可以省去自己执行定时任务重置 qps 和网关中累加 qps，坏处是引入了额外的依赖增加了系统的复杂度，需要考虑引入 `sentinel` 对系统原先模块造成的影响来衡量是否自己实现 qps 统计。

你提到合并请求来提升查询效率，直接用 `mysql` 连接池复用和 `mysql` 的连接不就可以了吗？

连接池可以通过减少创建和销毁与数据库连接的时间来提升查询效率，合并请求则在这基础上还减少了 SQL 解析、网络往返、事务提交等 SQL 执行层面的开销，降低数据库 IO 压力。

定时任务用的什么？

`XXL-job`，也可以用 `Spring` 自带的注解或者定时任务线程池，后两者简单易使用但功能相对单薄，而 `XXL-job` 虽然配置更复杂但有单独的 ui 界面可以在界面做配置，能和代码形成一定程度的解耦，同时日志记录详细，可以更方便的追踪任务的执行状态和结果。

哪些请求需要（可以）被视为高并发情况下要做合并的请求？

这个合并请求方案关键是将 `sql` 的 `where` 改成 `in` 来批量查询，因此类似获取用户信息、获取视频信息这种传参是实体 id 的接口都可以。

合并请求流程是怎么想到的？

考虑到项目的场景会涉及到高并发，因此提问 ai 后得到多条应对思路，从其中选择合并请求这一条，再自己慢慢完善合并请求的完整过程。

写这个流程时有遇到什么困难？

想单独请求转向合并请求的 qps 阈值测试方案和优化方案时相对有些困难，以及测试过程比较繁琐。

如果有上万个自定义对象卡在队列中对服务器负担很大，还能怎么优化吗？

这种高并发情况可以给每个用户信息设置 redis 的缓存，请求进来查询数据库前先查询是否 redis 中有缓存，如果有就直接用缓存，没有就查询数据库并且将查询出来的结果设置缓存，然后修改用户信息后修改对应用户的缓存。

这样对 redis 压力过大怎么办？

做数据统计，只缓存访问频率大于一定数量的用户信息。或者采用最近最少使用缓存淘汰策略，相比其他淘汰策略，用这个策略在保证减少 redis 缓存压力的前提下还能在高并发期间充分提高缓存命中率。

备注：缓存命中的意思是一次查询走缓存即可而不需要查询数据库

高并发下对同一个用户信息又读又写导致读操作读到了旧数据怎么办？

可以做读写锁，写的时候创建写锁，读的时候查询是否有写锁，没有写锁才能读，否则阻塞线程直到无写锁，写之前也先查询是否有写锁，有写锁也不会继续写，防止多个写的线程执行顺序不一导致数据修改的最终态不是预期的最终态。

这是用户修改信息的优先级比较高的情况，如果读取用户信息的优先级比较高，那读操作执行后对读的线程计数，写操作前还要查询是否有读的线程计数是否为 0，线程计数不为 0 则还有读的线程，就不让执行修改用户信息的写操作。

还有哪些可以应对高并发的方案？

整体思路一是在不改变现有处理能力的情况下拒绝超出处理能力范围的请求，对超出阈值的请求进行限流或者熔断降级处理

二是提升现有处理能力，增加服务器配置，部署多个实例进行负载均衡

然后优化慢 sql 和在合适的列上创建索引提升查询速度

分库分表将数据分散到多个数据库或表中读写分离来提升数据库的写入和查询速度

然后将热点数据缓存到 redis 中减少对数据库查询，减小数据库压力和增加响应速度，以及可以将一部分工作放到子线程中异步进行来提升请求的响应速度。

你提到了队列存储自定义请求对象，那你这个队列设置的容量是多少呢，怎么考虑的？

1200，查了资料后知道用户 2s 以内得到响应会感觉系统的响应很快，而项目定时任务的执行周期是 500ms，也就是说请求处理的最大延迟时间是 500ms，因此按 1.5s 为阈值去做 mysql 批量查询测试，当同时处理的请求数为 1200 时批量查询并把所有的用户信息都填装到对应的线程里用时为 1.5s。

唯一请求 id 是如何生成的？

用的 uuid 生成的，如果考虑 id 重复的可能性也可以优化，在原先的 uuid 后面加上请求的时间戳来进一步降低 id 重复的概率。

Completablefuture 有哪些常见用法？

执行有返回值的异步任务，执行无返回值的异步任务，阻塞当前线程直到调用 complete 方法（主线程中调用 get 方法阻塞的就是主线程，子线程中调用 get 方法阻塞的就是子线程）

如果现在我想用缓存的方式应对高并发，应该怎么办？

可以采取双级缓存的方案，核心原则是优先查缓存，缓存未命中再查数据库。

在查询链路里，第一步会先查本地内存缓存也就是一级缓存，如果命中直接返回结果，这一步是最快的。如果一级缓存没命中，再查分布式缓存也就是二级缓存，要是二级缓存命中了，会先把结果同步到一级缓存，相当于给本地预热，然后再返回。只有当两级缓存都没命中时，才会去查数据库，查到结果后，会把数据同时写入一级和二级缓存，再返回给前端。这样后续同个实例或其他实例的查询，就能直接走缓存了。

而在更新链路里，要重点保证缓存与数据库的一致性，采用先更数据库，再删缓存的策略，比如更新用户信息时，先把数据库里的记录更新完，再主动删除本地一级缓存和分布式二级缓存里的旧数据，而不是直接更新缓存，因为如果直接更新缓存，在高并发下可能出现多个线程同时更新，导致缓存数据覆盖错误的问题，删除缓存反而更安全，后续查询时自然会从数据库拉取最新数据并重建缓存。

怎么确保请求执行的顺序性？

项目中的合并请求是每隔一段时间批量查询许多请求需要的响应，然后填装到各自的线程中，结束各自线程的阻塞并返回前端，这种场景下是有可能出现后到的请求的响应先填装到所在的线程中，然后先返回前端的，且无负面影响，所以不需要保证请求执行的顺序性，如果要保证的话就给自定义请求对象加个时间戳字段，并在查询出用户信息后遍历自定义请求对象时按时间戳由小到大的方式进行遍历，保证先到的请求先处理。

综合性

请介绍整个系统后端的架构设计，有哪些模块以及各模块的作用和各模块之间的关系？

总共六个业务服务，一个公共服务，六个业务服务分别是网关服务，视频服务，聊天与大模型服务，消息服务，用户服务，搜索服务

网关服务

作为请求入口，接收请求后除非登录注册请求，其他请求一律经过鉴权后路由转向到其他服务，鉴权不通过则返回 401 状态码，登录后授权，定时刷新 token 权限

视频服务

视频的分片上传与断点续传，视频的点赞评论弹幕收藏创建收藏夹

聊天与大模型服务

一对一实时私聊，大模型文生文文生图文生 ppt

消息服务

点赞、评论、私聊、关注 up 动态消息的生成和消费，数据同步增量消息存到 redis

用户服务

用户信息查看编辑，用户个人主页权限查看编辑

搜索服务

推荐视频，视频和用户的条件搜索，关键字补全和高亮，数据同步执行

公共服务

存放可复用类，如 mybatis-plus 的特定类和 openfeign 接口类，全局异常处理器，mybatis-plus 自动填充器，泛型响应类，自定义序列化器

各模块之间的关系是通过 openfeign 互相调用。

什么是微服务架构？它有什么优点？为什么在项目中选择使用微服务？

微服务架构是一种将单个应用程序开发为一组小型、独立的服务的架构方法，每个服务都可以独立地进行开发、部署和扩展，这些服务通过轻量级通信机制（如 HTTP/REST）进行交互，共同构成一个完整的应用系统。

微服务架构的优点

高可扩展性，每个微服务可以根据其自身的负载需求独立地进行扩展。例如，在电商项目中，订单服务在促销活动期间可能需要处理大量订单，可单独对订单微服务进行扩展，增加服务器资源，而不会影响到其他服务，如用户服务、商品服务等。

敏捷开发与部署，各个微服务可以由不同的团队独立开发、测试和部署，加快了开发和迭代速度。不同微服务可以根据实际需求选择最合适的技术栈，提高开发效率。

高可靠性，当某个微服务出现故障时，只会影响到该服务本身，不会导致整个系统崩溃。系统的其他部分仍然可以正常运行，提高了系统的稳定性和可靠性。例如，在社交媒体系统中，若图片处理微服务出现问题，用户仍然可以正常浏览动态、发送消息等。

易于维护和升级，每个微服务的功能相对单一，代码量相对较小，维护起来更加容易。当需要对某个功能进行升级或修改时，只需要关注对应的微服务，降低了维护成本和风险。

可以应对复杂业务需求，将复杂业务拆分成多个简单的、可管理的微服务，每个微服务负责一个特定的业务功能，使项目结构更加清晰，便于理解和管理。

点赞功能的表是怎么设计的？

主键 id，点赞的视频 id，点赞者的用户 id，点赞的评论 id，点赞创建时间。点赞既可以是对视频也可以对用户，因此点赞的视频 id 和点赞的评论 id 都存储在表里，对视频的点赞则评论 id 为空。

你在项目中是如何设计库表的？可以从字段、索引、关联等方面回答？

根据有哪些实体、对实体的动作和实体与实体间的关系建的，如有视频实体和用户实体就建视频表和用户表，有对视频的点赞、评论操作就建立点赞表和评论表，有收藏夹和视频且是多对多关系就额外建一个收藏表做中间表。

为什么要做这个项目？做之前有调研过其他同类产品吗？

这个项目是在牛客上看到的开源项目，做这个项目是因为有用到目前主流 Java 企业开发技术栈如各大微服务组件 SpringBoot 这些，有相较于其他单体或微服务项目来说更加复杂的业务和流程，也有充足的可供扩展设计的空间，熟练掌握后能有效提升代码能力、理解需求能力和根据需求制定和抉择技术方案的能力，这样进入公司后所需适应时间就会比较短，公司的培养成本也会比较低。有调研过，综合比较最能综合提升多方面能力的还是这个项目，因

此选择了这个项目。

项目是你一个人做的吗？

有一个前端，前后端分离，我负责所有后端，他负责所有前端。

功能是如何划分到各服务的？

按几个重要的实体去划分，和权限相关的登录注册以及授权鉴权等功能都划到统一处理权限的网关服务，和用户相关的操作如修改用户信息、修改个人主页权限和关注用户等都划到用户服务，和视频相关的操作如上传点赞评论弹幕收藏等都划到视频服务，和消息队列中的消息有关的如各主题消息的生成与消费，未读消息数量和未读消息内容查询等都划到消息服务，和搜索结果相关的如关键字搜索、关键字补全、数据同步等都划到搜索服务，和聊天消息有关的如聊天会话的增删改查、聊天消息的增删改查和聊天消息的实时传递、和大模型的聊天等都划到聊天服务。

项目中遇到的最大的困难？

mysql 到 es 数据同步，涉及的组件比较多，流程也比较长，是把 mysql 的所有视频表、用户表的数据同步到 es 的视频索引和用户索引中....（然后把数据同步流程完整讲讲就可以了）

你的视频平台相对于 bilibili 的区别是什么？

数据量更小，功能没有 bilibili 那么多，但也加了一些 bilibili 没有的功能如集成大模型。

介绍下视频平台？

本项目仿照 bilibili，旨在提供一个前后端分离的视频平台，实现了多种方式注册登录、视频的上传、点赞、收藏、弹幕、评论、个人主页内容与权限的编辑查看、聚合搜索、私聊与大模型文生文、文生图、智能 ppt 等功能

为什么有这么多性能优化的思考？

觉得性能优化是实习工作中除去接口实现外，必不可少的步骤，因此在学习过程中有意识的设计和改进性能优化方案。

Mysql 表有哪些？

视频、视频数据、用户、点赞、点赞通知、评论、评论通知、收藏夹、收藏、弹幕、聊天消息、聊天会话、历史播放、用户权限。

如何记住这些表？

项目中的实体有视频、用户、收藏夹，因此有对应的视频用户收藏夹表，而对这些实体的操作有点赞、评论、弹幕、收藏、关注、播放，因此有对应的表，同时因为视频和收藏夹的关系是多对多，一个视频可以在多个收藏夹中，一个收藏夹也可以有多个不同的视频，有中间表，中间表即为收藏表，将视频数据存到表里可以减少每次查询视频时对点赞评论弹幕的聚合查询，因此有额外的视频数据表，用户可以设置自己的个人权限因此有用户权限表，有后台消息通知因此有点赞通知和评论通知表，用户可以创建会话和在某个会话中和其他用户聊天，因此有聊天会话表和聊天表。

项目中用到了哪些设计模式？

没有特意去写设计模式实现，但了解一些常用的设计模式

如工厂模式，工厂模式是一种创建型设计模式，其核心思想是将对象的创建和使用分离，把对象创建逻辑封装在一个工厂类中，让使用者无需关心对象具体的创建过程，只需向工厂请求所需对象即可

这样做能提高代码的可维护性和可扩展性，优点是解耦对象的创建和使用，使用者只需关注如何使用对象，无需了解对象的创建细节，降低了代码的耦合度，便于维护和扩展

当需要修改或新增产品时，只需在工厂类中进行相应的修改或扩展，不会对其他部分的代码产生影响

缺点是代码复杂度增加，引入工厂类会增加代码量和复杂度，特别是在简单场景下，使用工厂模式可能会显得过于繁琐，以及如果工厂类负责创建多种类型的产品，可能会导致工厂类的职责过重，违反单一职责原则。

对 ai 有多少了解？

当前比较火的方向是 ai 应用层开发，集成大模型 api 来实现各种功能，其中一个热门功能就是实现 agent 智能体，而 agent 代表着具备自主决策、环境交互和目标导向能力的智能系统，相比普通 ai 更接近人，有一套融合感知、规划、行动和学习的综合架构，有更强的自主性。

有对比过 ai 吗？

由于讯飞提供给开发者的 api 免费额度比较多因此用了讯飞的，就没有过多对比其他家提供给开发者的 api，但模型的使用有对比过，claude 和 genimi 模型答案生成质量比较好，写代码尤其强大，但使用需要翻墙或者去镜像网站，国产大模型如豆包 deepseek 这些使用简单，但是对应的生成质量不算特别理想

自我介绍一下？

面试官您好，我叫不吃香菜（就业版），本科就读于 xxx 大学 xxx 专业，研究生就读于 xxx 大学 xxx 专业（如果是本科生就换成就读于 xxx 大学 xxx 专业），做了一个有视频、用户、aigc 等功能的仿 b 站项目和一个有购买、寻找商户等功能的仿大众点评项目。

项目是怎么部署的？

地用 maven 插件打 jar 包，再将 jar 包传到 finalshell 上，在 finalshell 中执行命令 nohup java -jar out 后台执行服务并生成一个默认的日志文件，ps -ef grep 查看服务的 jar 包的端口号，kill -9 服务的端口号杀掉服务，rm -rf 删除旧的 jar 文件

为什么没上线？

因为服务器过期了，没钱续费

项目是怎么测试的？

测试的话就是使用 swagger 接口文档，运行程序后测试接口运行会不会报错，然后填写几个数据作为接口传参，看调用的接口得到的响应是否有误，这是功能测试，如果做性能测试就是用 jmeter 测量某个 qps 下的响应时间，多次测量得到平均值减小误差。

现在有一个接口，我觉得这个接口的耗时非常高，我想把它优化一下，你可以给一些优化耗时的方法吗？

接口耗时高的优化可以从多个层面入手，首先可以从数据传输角度考虑，比如减少不必要的

返回字段，只返回接口实际需要的数据，避免传输冗余信息增加网络开销。也可以对返回数据进行压缩，降低数据传输量。

其次是缓存策略，对于接口中频繁访问且不常变更的数据，可以引入缓存机制，比如使用 Redis 等缓存工具，将数据提前存储在缓存中，后续请求直接从缓存获取，减少对数据库或后端服务的直接访问。

然后是数据库层面的优化，如果接口耗时主要源于数据库操作，那可以检查并优化 SQL 语句，避免全表扫描，通过建立合适的索引提升查询效率。也可以考虑分库分表，将大量数据分散存储，减少单表数据量，提高查询速度。

另外，还可以从接口本身的实现逻辑入手，简化复杂的业务处理流程，避免不必要的计算和调用。对于耗时较长的操作，可采用异步处理的方式，让接口快速返回，后续通过回调等方式处理结果。

同时，服务器和网络环境也可能影响接口耗时，合理配置服务器资源，比如增加 CPU、内存等，或使用负载均衡将请求分发到多个服务器，避免单点压力过大。优化网络链路，减少网络延迟，也能有效提升接口响应速度。

弹幕是怎么做的？

后端维护弹幕表，每次用户发送弹幕后往数据库中插入一条弹幕记录，用户进入页面时从弹幕表中查数据并返回前端，弹幕表的列分别是视频 id，弹幕在视频中的时间戳，弹幕内容，弹幕发送人 id，弹幕的创建时间。

项目的 jdk 用的几？

Jdk8。

技术选型

项目中为什么要使用 Redis？

Redis 的应用场景有缓存热点数据减小数据库压力、存放一些无需持久化的数据、排行榜/计数器、发布/订阅、分布式锁、队列、存储地理位置信息，而项目中有数据同步的模块需要缓存增量信息，因此用到了 redis。

项目中为什么要使用 Websocket？

Websocket 适用于需要保持服务端与客户端双向通信或服务端主动通知客户端或需要低延迟的场景，而项目中私聊需要实时也就是低延迟的消息通信，同时私聊架构是客户端向服务端发 websocket 消息并由服务端主动通知另一个客户端，因此采用 websocket。若要采用 sse 技术则需要修改前后端交互格式，前后端建立 sse 连接，一个客户端向另一个客户端发送消息采取向服务端发送请求服务端再通过 sse 连接向另外一个客户端主动推送消息的形式。

项目中为什么要使用 Minio？

Minio 适用于有存储文件需求的场景，文件的上传下载性能更高，且可以方便的登录控制台查看和编辑文件，而项目中若采用 mysql 保存文件流会出现查询速度慢、容易出现数据库 I/O 瓶颈、存和取文件还需要额外转换格式等问题，因此选择对象存储服务，minio 相较 oss 来说定制性更强，数据主权和控制权更大，但当前项目暂时没有这方面的需求，选择 minio 的

关键原因是可以省钱。

项目中为什么要使用 Mybatis-plus?

便捷执行单表增删改查

减少代码量

代码可维护性高

项目中为什么要使用 Nacos?

微服务间通信需要一个服务发现与注册中心来方便服务间的互相调用

项目中为什么要使用 Gateway?

可以作为唯一请求入口，统一鉴权与授权实现单点登录

项目中为什么要使用 es 做搜索而不使用 mysql 的模糊搜索?

因为 es 的 api 有比较成熟的搜索实现，对每个文档去匹配打分，然后只获取分数高的文档，相比 mysql 的模糊搜索精确度更高。

项目中为什么要使用 XXL-JOB?

XXL-JOB 是一个分布式定时任务框架，天然支持分布式，可以页面配置定时任务，与代码解耦，且可以很方便查看执行的任务的日志，也能看到执行是否成功和失败，还有统计功能，而项目中有数据同步、QPS 重置、定期批量处理合并请求需要定时任务，因此使用了 XXL-JOB。

项目中为什么要用 rocketmq?

因为消息队列是企业常用技术，想把这一块熟练掌握，正好项目中有些流程可以解耦，就会用到消息队列，而 Rocketmq 具备高可用性，丢失数据的风险较小，有高吞吐与低延迟，性能较好，且原生支持事务消息、延迟消息、重试队列，覆盖挺多业务场景的个性化需求，无需额外开发。且作为阿里开源的中间件，文档完善、社区活跃，遇到问题能比较快找到成熟的解决方案。

项目中 nacos 做配置中心是怎么做的?

没特地用 nacos 做配置中心，只用作了服务注册与发现中心，但是知道配置中心，用于集中管理微服务的配置信息。允许开发人员在重新部署微服务的情况下，动态地修改配置参数。配置中心提供了一个统一的配置管理平台，使得配置的修改和分发更加便捷和高效，同时也保证了配置的一致性和安全性。

xxl-job，了解实现原理吗?

XXL-Job 是一款分布式任务调度框架，其核心实现原理基于调度中心与执行器的分离架构。调度中心负责管理任务调度信息，按照配置的调度规则触发任务，并通过注册中心获取执行器的地址列表，采用负载均衡策略选择合适的执行器发送任务执行指令。

执行器则是任务的实际运行载体，它会主动向注册中心注册自己的地址，接收调度中心的指令并执行对应的任务逻辑，执行完成后将结果反馈给调度中心。

这种架构通过注册中心实现了调度中心与执行器的解耦，支持执行器的动态扩缩容。同时，框架内置了任务失败重试、任务超时控制、日志追踪等功能，确保任务调度的可靠性和可追

溯性。

在通信层面,调度中心与执行器之间通常采用 HTTP 协议进行交互,简化了部署和集成成本,也便于跨语言扩展执行器。

Xxl-job 是分布式部署的吗?

不是,只做了本机的单体部署。